

Explicação Técnica de Códigos

Arquitetura e Lógica do Projeto To-Do List em C#

Este guia técnico detalha a estrutura de código, classes e métodos fundamentais utilizados no desenvolvimento da sua aplicação de gerenciamento de tarefas (**ToDoListConsole**). O objetivo é solidificar o entendimento de como o C# manipula dados e interage com o sistema de arquivos.

1. O Modelo de Dados: A Classe Tarefa

Em Programação Orientada a Objetos (POO), usamos classes para representar coisas do mundo real. A classe Tarefa define quais propriedades cada item da nossa lista possui.

```
public class Tarefa
{
    public int Id { get; set; }
    public string Descricao { get; set; }
    public bool Concluida { get; set; }
}
```

Explicação dos Componentes:

- `int Id`: Um identificador único numérico para cada tarefa, facilitando a busca, exclusão e alteração de status.
- `string Descricao`: O texto que descreve o que precisa ser feito.
- `bool Concluida`: Um valor booleano (verdadeiro/falso) que indica se a tarefa já foi finalizada.

2. Persistência com JSON (Salvar e Carregar)

Para evitar que os dados sumam ao fechar o console, utilizamos a biblioteca `System.Text.Json`. Ela transforma nossos objetos C# em texto estruturado e vice-versa.

Método para Salvar Dados (Serialização):

```
string jsonText = JsonSerializer.Serialize(listaDeTarefas);
File.WriteAllText("tarefas.json", jsonText);
```

Como funciona: O método `Serialize` converte a lista em uma string formatada em JSON. Logo após, `File.WriteAllText` cria (ou sobrescreve) o arquivo `tarefas.json` no disco com esse texto.

Método para Carregar Dados (Desserialização):

```
if (File.Exists("tarefas.json"))
{
    string jsonText = File.ReadAllText("tarefas.json");
    listaDeTarefas = JsonSerializer.Deserialize<List<Tarefa>>(jsonText);
}
```

Como funciona: Primeiro verificamos se o arquivo existe para evitar erros. Se existir, lemos o texto completo e o `Deserialize` reconstrói a estrutura original de `List<Tarefa>` na memória para podermos manipulá-la via código.

3. O Loop Principal do Console (Menu Interativo)

Para manter o programa rodando continuamente até que o usuário decida sair, usamos uma estrutura de repetição `while(true)` combinada com um bloco de escolha `switch`.

```
while (true)
{
    Console.WriteLine("1 - Adicionar Tarefa | 2 - Listar | 3 - Sair");
    string opcao = Console.ReadLine();

    switch (opcao)
    {
        case "1":
            // Lógica para Adicionar
            break;
        case "2":
            // Lógica para Listar
            break;
        case "3":
            return; // Encerra o loop e o programa
        default:
            Console.WriteLine("Opção inválida!");
            break;
    }
}
```

Por que usamos o Switch Case?

O `switch` avalia o valor contido na variável `opcao` de forma muito mais limpa e legível do que usar múltiplos blocos de `if / else if` encadeados. Cada `case` define o bloco de código que deve ser executado para aquela escolha específica.

4. Customização Visual da Interface CLI

Para tornar o console profissional e de fácil leitura, utilizamos propriedades nativas da classe estática `Console` do .NET:

- `Console.ForegroundColor`: Modifica a cor da fonte do texto a ser impresso na tela.
- `Console.BackgroundColor`: Altera a cor de fundo do texto subsequente.
- `Console.Clear()`: Limpa todo o histórico de texto visível na janela do console, permitindo redesenhar o menu de forma organizada e estática a cada ação do usuário.